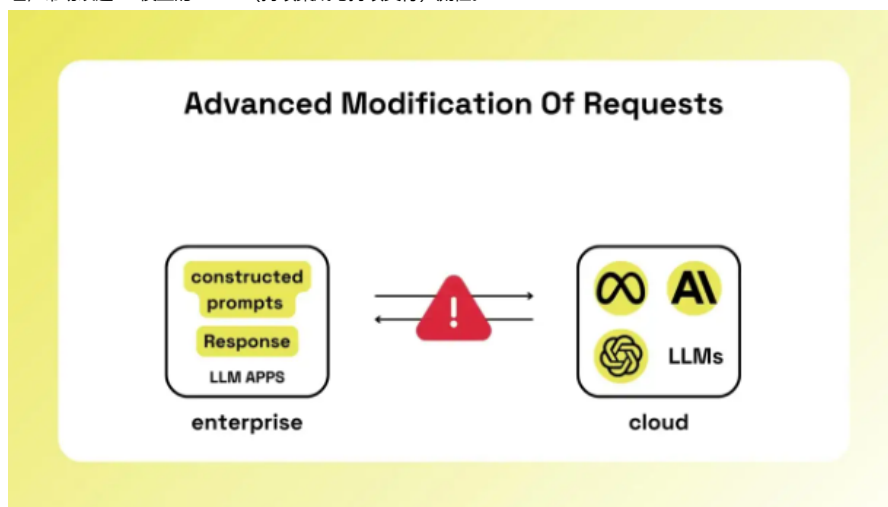


如何同时接入多个AI大模型？LLM代理/LLM网关的应用

自从 OpenAI 推出 ChatGPT 开始，AI 领域经历了巨大的变化。针对不同的使用场景进行了优化，OpenAI 提供了多个版本的 AI 模型。一些模型响应速度更加的快，还有一些则更加注重精度。令我，还有许多其他厂商和模型可供选择，比如 Hugging Face 上近两万个开源模型。在如此多样化的 LLM（大语言模型）生态中，“用一个 LLM 就解决所有问题”的设计思路似乎已经难以满足实际需求。

为此，开发者们逐渐采用更加灵活的策略来管理大模型的请求，其中主要有两种：**专家混合型****（Mixture of Experts, MoE）** 和 **LLM 代理（LLM Proxy）或 LLM 网关（LLM Gateway）**。

下面将介绍 LLM 代理或LLM网关在 AI 驱动应用中的通用设计模式，同时也探讨一下如何利用开源工具构建自定义的 LLM 代理，帮助改进 AI 模型的 CI/CD（持续集成与持续交付）流程。



什么是 LLM 代理/LLM网关/AI网关？

LLM 代理和 LLM 网关、或者说 AI网关 是一种架构解决方案，即通过抽象化访问大语言模型（LLM），在应用程序与 LLM 之间充当中间层。

这个方案不仅能够简化 API 集成，还借鉴了传统代理服务器的优势，例如日志记录、监控、负载均衡等功能。通过这种架构，可以为不同任务选择“最合适的模型”，而无需在应用程序层面进行大量代码修改，避免了“一刀切”的模型使用假设。

LLM 代理/LLM网关解决的挑战

在生产环境中部署 LLM 时，主要解决的需求是如何实现访问来自不同供应商的多个模型。这种设计模式通常被称为“多 LLM”（Multi LLM）。随着企业从 GPT-4o 切换到 GPT-4o-mini，或尝试其他选择如 Anthropic，直接对接每个 LLM 供应商会变得非常复杂且低效。LLM 代理可以有效解决这一问题，通过统一的接口简化模型切换和管理。

```
import os
from anthropic import Anthropic

client = Anthropic(
    # This is the default and can be omitted
    api_key=os.environ.get("ANTHROPIC_API_KEY"),
)

message = client.messages.create(
    max_tokens=1024,
    messages=[
        {
            "role": "user",
            "content": "Hello, Claude",
        }
    ],
    model="claude-3-opus-20240229",
)
print(message.content)
```



```
from openai import OpenAI
client = OpenAI()

completion = client.chat.completions.create(
    model="gpt-3.5-turbo",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Hello!"}
    ]
)
```



如上图所示，每个供应商的 API 可能略有不同，当你需要在不同模型之间切换时，往往需要部署新版本的代码。也就是说你需要了解每个AI供应商的 API，并具备一定的专业知识。

然而，随着AI供应商的越来越多（包括各种开源的选择），这种接入方式显然困难且低效。

理想情况下，你可以通过统一 API 访问的方式来解决多个大模型的调用，这样不仅模型之间可以快速切换，而且无需修改应用程序的代码。

虽然这种实现也可以通过 LangChain 来管理，但目前，LangChain 是在应用层实现这一功能的，而 LLM 代理则允许你可以从一个平台上对多个应用进行统一配置。

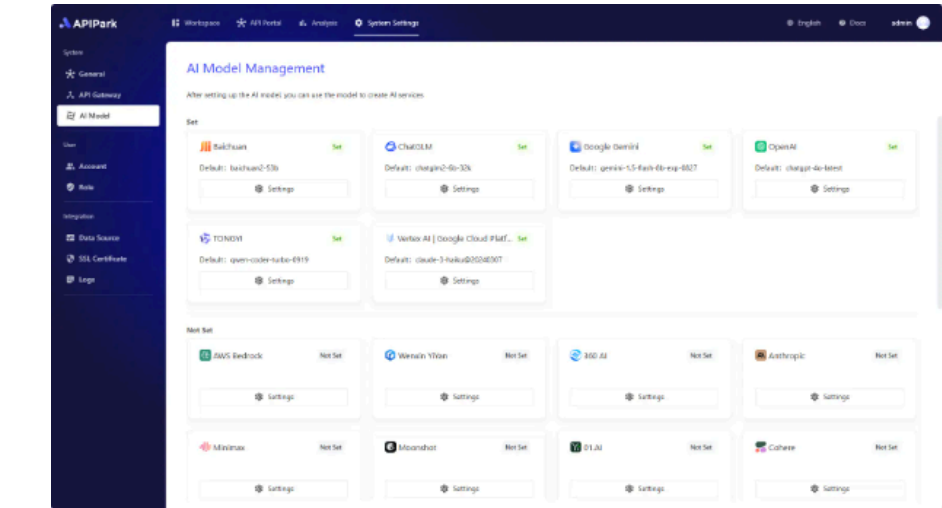
开源LLM网关解决方案

以下是使用APIPark 配置 LLM 供应商的示例，Github仓库

设置AI模型供应商

在开始创建 AI 服务之前，首先需要配置 AI 模型供应商。APIPark 支持多种 AI 模型，包括 OpenAI、Anthropic、AWS Bedrock、Google Gemini等。配置供应商后，就可以选择不同的模型来创建 AI 服务，并在 APIPark 中统一管理所有 AI 服务的授权信息和成本统计。

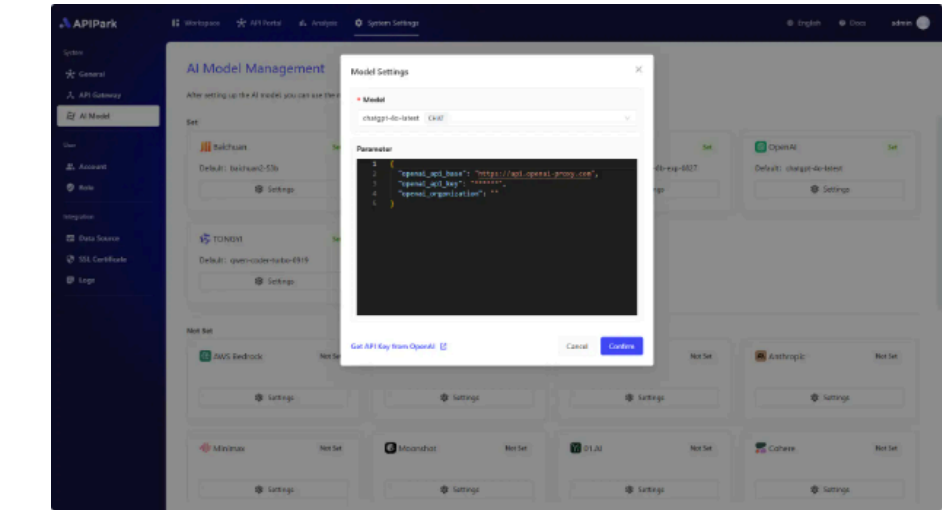
进入 系统设置 模块，在侧边栏中选择 AI 模型管理，在列表中可以看到 APIPark 支持的所有 AI 供应商。



以接入 OpenAI 为示范，点击设置按钮，在弹窗中：

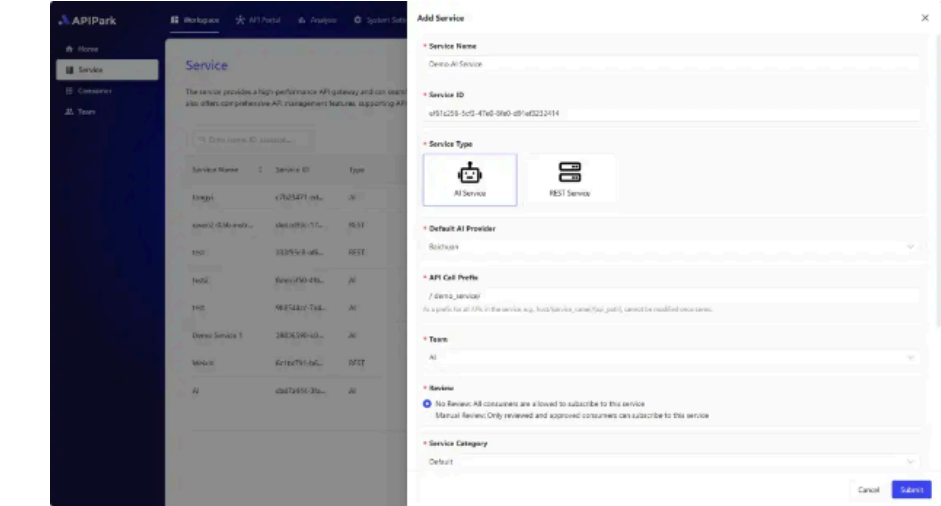
- 1) 选择 默认的 AI 模型：后续在 AI 服务中创建 API 时，系统会自动为 API 设置默认的 AI 模型，减少用户操作的次数。
- 2) 填写 供应商配置：每个供应商有不同的配置信息，系统会自动根据你选择的供应商来生成所需的配置信息。

配置信息一般可以在供应商的 Open API 管理后台获得，你可以在弹窗左下角找到快速跳转到供应商官网的按钮。



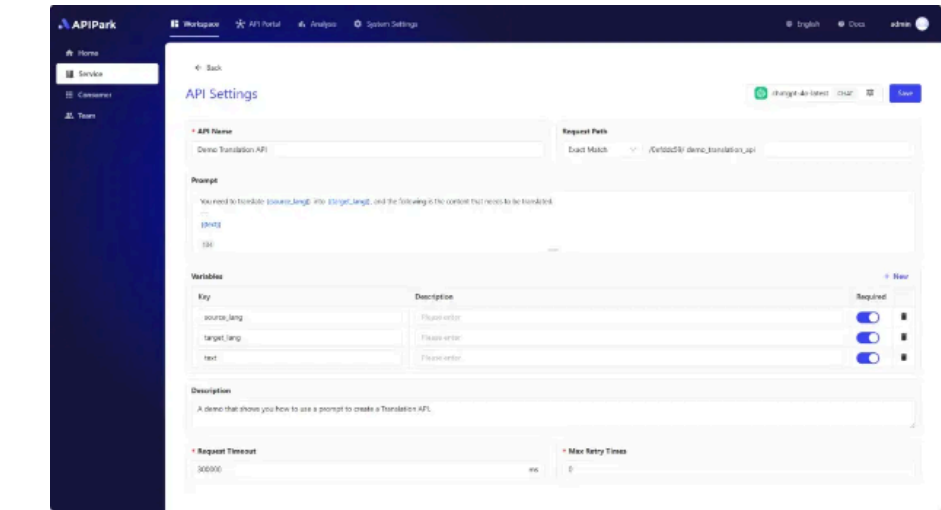
创建并发布 AI 服务

1) 进入 **工作空间** 模块，在侧边栏中选择 **服务**，然后 **创建服务**，填写 **AI 服务的基本信息、配置管理权限和订阅审核形式**。

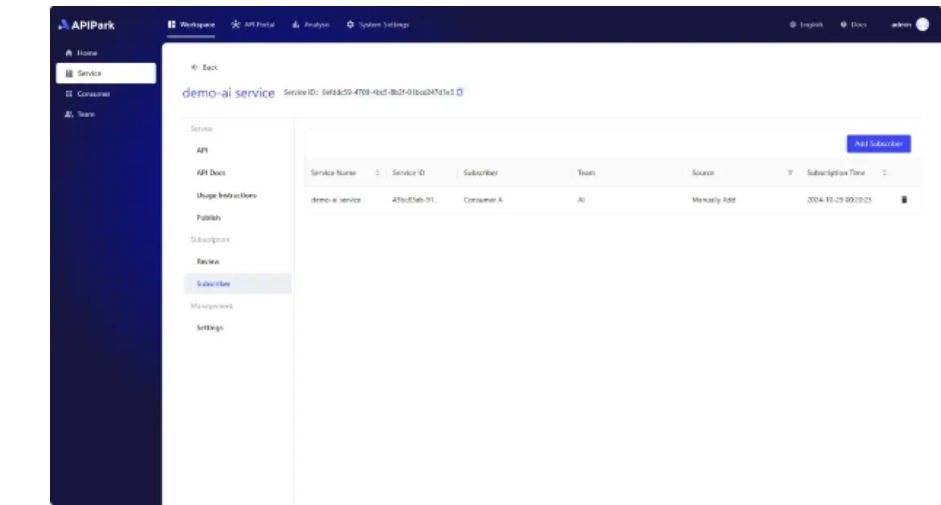


2) 创建服务之后，APIPark 会自动在服务里创建一个默认的 **聚合 API**（Unified API），你可以直接通过这个聚合 API 来调用 AI 服务。

以往企业调用 AI 过程中，涉及到 Prompt 提示词都是采用硬编码的形式写在系统中。在 APIPart 上你可以对每个 LLM API 的 Prompt 独立管理，**你可以将 Prompt 提示词和 AI 模型组合成自定义的 AI API**。



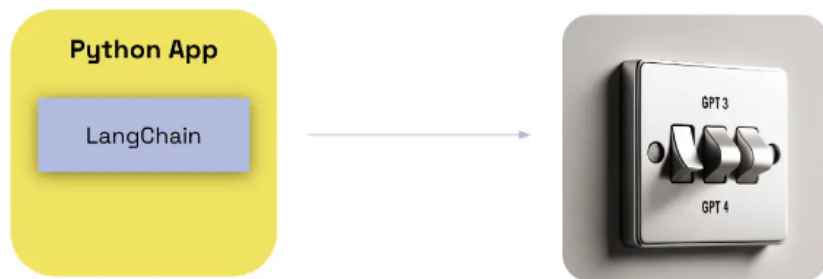
3) 将配置了 Prompt 的 AI API **发布到 API 门户上供大家订阅**，你可以给 API 配置审核流程，团队内外的开发者需要等待审核通过之后才能调用，你还可以在列表中看到所有订阅该服务的 **消费者**。



监控AI 应用程序

除了统一 AI 访问接口以外，让代码变得更加通用化 适应 AI 引擎的变化也是至关重要的。很多情况下，你可以将 AI 应用程序的代码作为围绕 AI 模型的一组包装函数，而且这些函数是可以替换的。

基本上，相同的 LangChain 代码可以用来构建一个聊天机器人（比如本文中的示例），然后你只需通过配置更换模型，就能让你的聊天AI机器人变得更智能更高效。这种切换就像按一下切换开关一样简单。

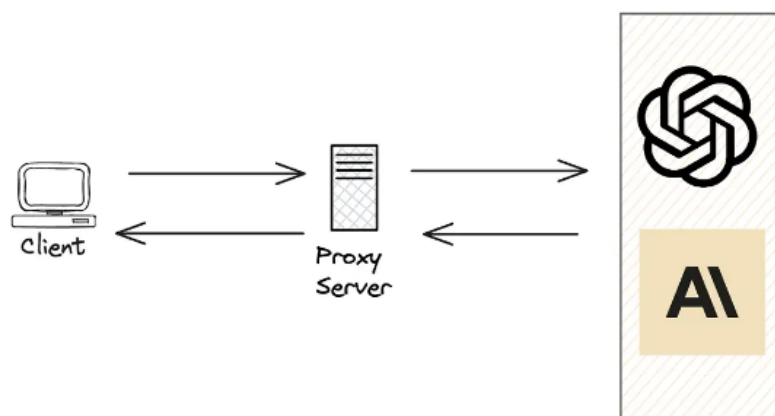


要知道应该选择哪些模型，你需要进行优化，而实现这一目标的关键在于高效的监控。

LLM 代理在这一方面发挥了重要作用，它提供了记录和监控与模型交互的工具。这包括跟踪延迟、Token 使用量和响应时间等数据，帮助组织根据性能指标优化其应用程序。

LLM代理的实际应用

代理是LLM代理的核心组成部分，充当客户端与LLM供应商之间的中介。它们使组织能够将请求路由到不同的模型，并无缝处理故障转移场景。LLM 网关通过提供统一的接口与多个LLM进行交互，从而简化了LLM API调用。这种方法减少了代码更改的需求，并支持模型之间的动态切换。

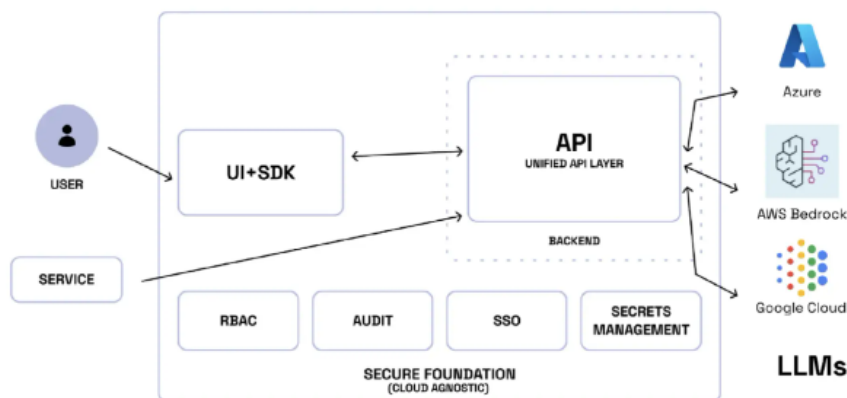


LLM代理服务器的示意架构

开源解决方案和自定义集成

企业组织如何利用LLM代理在AI和LLM API之上提供生产级功能。LLM 网关提供了一个用户界面，用于集成和测试不同的LLM，支持自定义扩展，并启用全面的日志记录和监控。这种灵活性使组织能够评估各种模型和提示，针对其特定用例进行优化。

下图是LLM网关如何在Google Cloud Kubernetes引擎上部署LLM代理，然后从应用程序向其发出LLM调用。

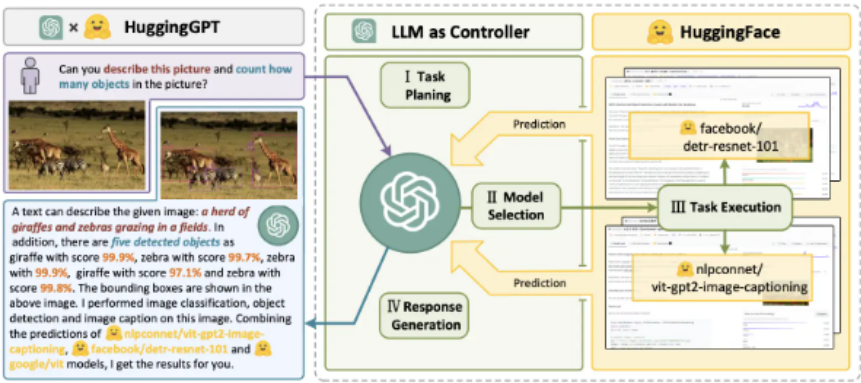


智能路由：选择代理还是 MoE？

正如前面提到的，LLM 网关的一大优势在于其能够智能管理对 LLM API 的访问。例如，当用户请求将英语翻译成西班牙语时，一个简单的 LLM 路由器可以识别出，这个请求更适合由专门为翻译任务微调的模型处理，而无需调用像 GPT-4 这样昂贵的基础模型。

Hugging Face 在其“Hugging GPT”中提出了这一概念，称之为“模型路由器”（Model Router）。

这种路由方式可以在组织层面实现一次性部署，而无需为每个单独的应用程序重复配置。然而，与这种路由方式相比，还有另一种竞争性技术——**专家混合（Mixture of Experts, MoE）**可能更加高效，因为 MoE 将路由层集成为 LLM 的一部分，在性能和资源分配上更具优势。



来源 HuggingGPT：使用 ChatGPT 及其在 Hugging Face 的伙伴解决 AI 任务

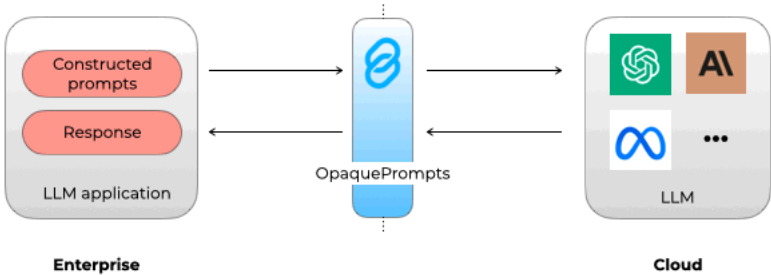
可扩展的 LLM 代理部署

到目前为止，本文介绍了将 LLM API 集中化的优势，但这种方法也可能带来一些潜在问题。为应对日益增长的流量并避免瓶颈，LLM 代理必须具备良好的可扩展性。

通过容器化的代理，可以在 Kubernetes 集群中部署代理服务，从而根据需求实现水平扩展。这种方式确保代理服务器能够在不影响性能和可靠性的情况下，处理大量的请求。下图展示了基于 Kubernetes 的可扩展 LLM 代理的一般架构。

添加额外的安全性、隐私和合规性

在集中访问LLM并添加日志记录和监控后，LLM网关可以通过集中访问控制、秘密管理和日志记录来增强安全性。它们还通过允许组织在向LLM提供者发送请求之前掩盖敏感信息来促进遵守数据隐私法规。一些解决方案，如opaque.co，使用小型内部LLM或神经网络来识别和删除个人可识别信息（PII），确保敏感数据保持在组织内部。



不透明 Prompt 确保个人信息不会泄露

写在最后

LLM代理是AI应用中的一种新兴设计模式，帮助组织管理生产中LLM应用的访问、安全和监控。通过集中访问、实现模型之间的网络级切换，并提供日志记录和监控工具，组织可以更细致地控制其AI工作负载，同时减少新LLM集成所需的努力。日志记录和监控还帮助维持对成本和合规性的控制。随着LLM的使用持续增长，LLM代理在确保高效和安全的部署中将变得越来越重要。然而，有人质疑这种设计模式是否会与基于专家混合方法的模型相比同样高效。